Taller de investigación aplicada:
Sistemas RAG con Milvus y LLM locales

Pontificia Universidad Javeriana · Maestría de Inteligencia Artificial

Ing. Luis Gabriel Moreno Sandoval, PhD · Guía de laboratorio y evaluación · Octubre de 2026

Reto. Diseñar, implementar, desplegar y evaluar de forma reproducible un asistente RAG sobre un corpus real de prensa digital y publicaciones de Bluesky. Cada equipo investigará qué LLM local pequeño produce mejores respuestas con igual evidencia recuperada y condiciones de evaluación. No se permiten servicios de pago ni dependencias obligatorias de APIs comerciales.

1. Datos suministrados por el docente

El manifiesto *Documentos preparados para RAG* describe el archivo `documentos_rag.parquet`, que será distribuido por separado. Contiene **169.248 documentos padre y 33 columnas**, entre ellos 64.670 registros de `bluesky_posts` y 104.578 de `url_crawls`. No incorpora fragmentos ni embeddings y no representa un índice vectorial.

Campos esenciales: `document_id`, `text`, `collection_name`, `title`, `url`, `source`, `language`, `created_at`, `text_sha256`, `word_count` y `char_count`. El manifiesto reporta 14.771 filas adicionales de texto repetido. Los campos de adquisición no constituyen etiquetas semánticas validadas.

- Verificar esquema, nulos, unicidad de IDs, distribución por fuente, calidad de URLs y hash publicado, sin alterar silenciosamente el archivo.
- Definir una muestra reproducible: piloto de 500–2.000 documentos, después 10.000 y, si el hardware lo permite, el corpus completo.
- Preservar el texto original y los offsets; heredar procedencia en cada fragmento. No publicar el Parquet en GitHub sin autorización.

2. Arquitectura de referencia

La figura define los componentes y sus responsabilidades. **Milvus es el único motor de recuperación vectorial**; **Ollama** permite intercambiar los modelos locales sin modificar el recuperador. **Streamlit o un frontend propio** utiliza la API y muestra respuesta, fuentes y tiempos. La evaluación toma datos tanto de recuperación como de generación.

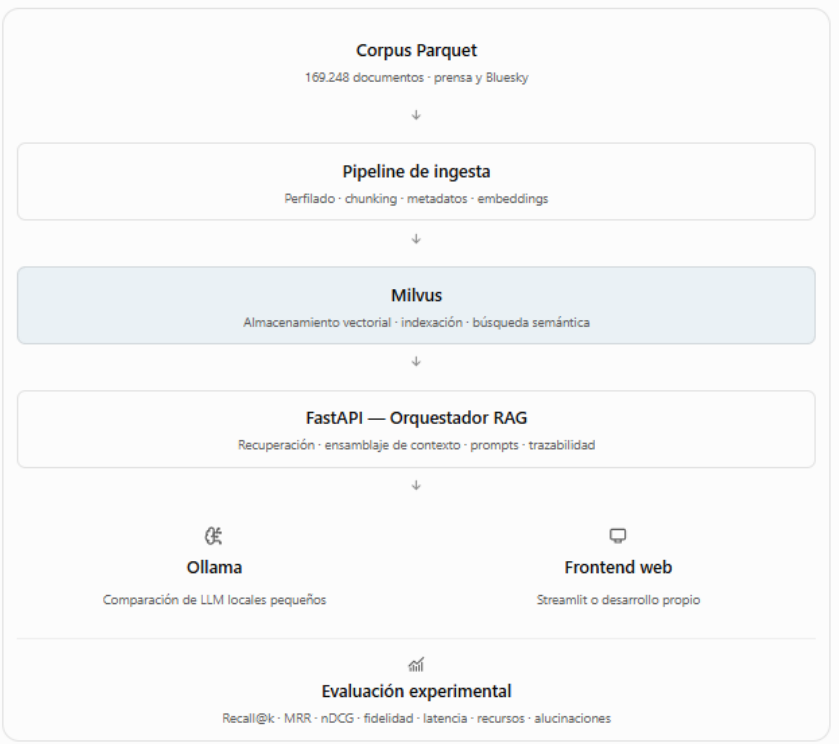


Figura 1: Arquitectura RAG de referencia (reproducción fiel del diagrama suministrado): corpus, ingesta, Milvus, FastAPI, Ollama, frontend y evaluación experimental.

Decisión de diseño: se debe documentar si el encoder de embeddings reside en la API, en un servicio separado o como trabajo de ingesta, así como el mecanismo para mantener idéntica la codificación de consultas y documentos. En Docker Compose, Milvus puede requerir servicios auxiliares como etcd y MinIO; son infraestructura, no motores alternativos de búsqueda.

3. Objetivos y tecnologías obligatorias

- Justificar una arquitectura desacoplada de ingesta, segmentación, embedding, almacenamiento, recuperación, generación, interfaz y evaluación.
- Desplegar un proyecto reproducible mediante un único `docker-compose.yml` en la raíz, con imágenes o Dockerfiles, `.env.example`, volúmenes y README.
- Implementar **Milvus standalone** como único vector DB; Python con PyArrow/Pandas y sentence-transformers o encoder abierto y local; **FastAPI** con `/health`, `/retrieve` y `/ask`.
- Implementar una interfaz web funcional con **Streamlit o desarrollo propio**, que permita elegir modelo, preguntar, ajustar `k` y examinar fuentes.
- Ejecutar mediante **Ollama** al menos tres modelos generativos locales pequeños sin cuotas pagas. Posibles candidatos (sujetos a disponibilidad y licencia): `qwen2.5:1.5b`, `llama3.2:1b`, `llama3.2:3b`, `gemma2:2b`.

4. Desarrollo técnico obligatorio

1. **A. Exploración y calidad.** Analizar fuentes, longitudes, idiomas, duplicados, fechas, metadatos, sesgos de recolección y distribución de la muestra. Presentar evidencia cuantitativa.

2. **B. Segmentación.** Diseñar y comparar dos alternativas de chunking, considerando tokens, solapamiento, offsets, IDs estables y deduplicación con trazabilidad de todas las procedencias.

3. **C. Embeddings.** Elegir y justificar un encoder multilingüe, dimensión, normalización, métrica de similitud y prefijos de consulta/pasaje cuando apliquen. Demostrar vecinos semánticos y analizar sensibilidad.

4. **D. Milvus.** Definir colección, campos, índice, filtros, métrica, configuración y estrategias de reindexación idempotente; verificar conteos, persistencia y recuperación.

5. **E. Orquestación RAG.** Separar recuperar y generar, versionar prompts, controlar presupuesto de contexto, exigir citas y permitir abstención cuando no exista evidencia suficiente. Registrar parámetros de cada ejecución.

6. **F. API e interfaz.** Presentar pregunta, LLM, respuesta, tiempo total, evidencias y enlaces; contemplar errores, estados de carga y elección dinámica del modelo.

7. **G. Operación.** Publicar código y contenedores, instrucciones desde cero, comandos de carga y descarga de modelos, pruebas, persistencia, logs y límites de CPU, RAM y GPU.

5. Investigación y diseño experimental

Pregunta: ¿qué modelo local pequeño ofrece el mejor equilibrio entre exactitud factual, fidelidad a fuentes, abstención apropiada, latencia y recursos en este corpus? Cada grupo formulará una hipótesis y un protocolo que permita refutarla.

Experimento principal: conservar constantes corpus, muestra, chunks, encoder, embeddings, colección Milvus, índice, `top-k`, contexto recuperado, instrucciones de prompt y parámetros compatibles; cambiar solo el LLM y documentar cuantización. Cuando una configuración no sea idéntica entre modelos, explicitar la amenaza a la validez.

Experimentos de ablación: variar de a una sola dimensión (`k`, tamaño de chunk, encoder, estrategia de contexto). Ante cambios de encoder, crear otra colección y regenerar sus vectores. Realizar al menos tres ejecuciones por condición cuando resulte viable; distinguir arranque en frío, cachés y calentamiento. Registrar hardware, semilla, versiones, tokens, memoria y fecha.

6. Validación reproducible

Construir un **gold estándar con al menos 40 preguntas** representativas de ambas colecciones, diversos idiomas y fechas, consultas directas y comparativas, casos ambiguos y preguntas sin respuesta documental suficiente. Cada pregunta debe incluir documentos/fragmentos relevantes, referencia verificable, tipo de consulta y criterio de respuesta. Separar conjuntos de desarrollo y prueba; no asumir que palabras clave de recolección constituyen verdad de referencia.

Dimensión	Evidencia mínima
Recuperación	Recall@k, MRR, nDCG@k, hit@k; inspección de resultados, duplicados y filtros.
Generación	Corrección, relevancia, faithfulness/groundedness, calidad de referencias y abstención correcta.
Rendimiento	Latencias p50/p95 (recuperación/generación), tokens por segundo, RAM, VRAM/CPU y tasa de errores.
Robustez	Alucinaciones, referencias incorrectas, fechas, discrepancias entre fuentes y resistencia a prompt injection en documentos.

Reproducibilidad	Configuración y versiones fijadas, anotaciones, resultados sin procesar CSV/JSONL, scripts y gráficos.

Emplear evaluación humana con rúbrica explícita y dos evaluadores en una submuestra. Cualquier *LLM-as-a-judge* será complementario, no una verdad absoluta. Explicar casos donde los modelos discrepan, aciertan sin evidencia, inventan información o se abstienen correctamente.

7. Entregables

- **Repositorio Git completo**: API, frontend, scripts de ingesta y embeddings, esquema/índices Milvus, evaluador, tests, Dockerfiles, `docker-compose.yml`, `.env.example`, documentación de arquitectura y scripts de inicialización. Datos restringidos y pesos de modelos se excluyen del repositorio.
- **README para replicación**: requisitos, instalación sin pago, carga del Parquet, elección y descarga de LLM, indexación, inferencia, comandos, evidencias de funcionamiento y solución de problemas.
- **Informe de investigación en PDF**: problema, hipótesis, alternativas y decisiones de arquitectura, diagrama propio, método, experimentos, resultados cuantitativos, análisis de errores, limitaciones, conclusiones y referencias.
- **Artefactos de evaluación**: gold estándar, anotaciones, configuraciones, resultados crudos, gráficas, tablas y trazas sin fabricar resultados.
- **Demostración**: preguntar a la interfaz, recuperar evidencias, alternar al menos tres LLM, mostrar tiempos e interpretar un caso de error.

8. Rúbrica de calificación (100 puntos)

Criterio	Puntos
Perfilado, integridad y análisis del corpus	10
Arquitectura y decisiones justificadas	15
Chunking, embeddings, trazabilidad y Milvus	20
API, interfaz, contenedores y despliegue reproducible	15
Diseño experimental y gold estándar	15
Comparación de LLM, métricas y análisis crítico	20
Documentación, pruebas y calidad del repositorio	5
Total	**100**

9. Repositorio demostrativo y alcance

Se suministra `Repositorio_Demo_RAG_Milvus.zip` como punto de partida con Docker Compose, API, frontend y un ejemplo de ingesta. No resuelve el taller ni contiene un benchmark. Los equipos deben adaptar, probar y ampliar el código. Ruta esperada del archivo real: `data/documentos_rag.parquet`. Inicio sugerido: copiar `.env.example` a `.env`, ejecutar `docker compose up -d --build`, descargar los modelos elegidos con `docker compose exec ollama ollama pull MODELO` y ejecutar la ingesta piloto documentada en el README.

Boilerplate oficial del curso: el repositorio base que se dejará como *boilerplate* para el proyecto RAG es https://github.com/puj-nlp/boilerplate_rag. Para el corpus, se distribuirá vía la librería `open_corpus_co_es` que esta en el url https://puj-nlp.github.io/open_corpus_co_es/; el dataset del taller se publica con el nombre `dragons_rag_2026_10_19`. Ejemplo mínimo de descarga y carga:

```
%pip install --upgrade --no-cache-dir open_corpus_co_es

from importlib.metadata import version
from open_corpus_co_es.loader import list_corpus, load_corpus
from open_corpus_co_es.downloader import download_corpus

print("Versión:", version("open_corpus_co_es"))

nombre = "dragons_rag_2026_10_19"
catalogo = list_corpus()

print("Entradas habilitadas:", len(catalogo))
assert nombre in catalogo, "El entorno todavía usa un catálogo anterior"

download_corpus(nombre, force=True)
datos = load_corpus(nombre)
print("Documentos:", len(datos))
```

Nota: el material compartido como descripción del corpus es el manifiesto Markdown, no el archivo Parquet. La ejecución Docker integral requiere verificación en el hardware del estudiante; recursos limitados pueden exigir procesar una muestra. Las licencias y requisitos de los modelos deben verificarse antes de su uso. **Documentación sugerida:** https://milvus.io/docs/, https://docs.ollama.com/, https://fastapi.tiangolo.com/, https://www.sbert.net/, https://docs.streamlit.io/.